

Chapter 27 — Topological Sort

0. What You Will Learn in This Chapter

Topological Sort is a graph pattern for ordering tasks with dependencies.

It applies to **directed acyclic graphs**, also called DAGs.

A topological ordering is an order of nodes such that every prerequisite appears before the thing that depends on it.

Example:

A must happen before B

B must happen before C

Valid order:

A, B, C

By the end of this chapter, you should understand:

- What topological sort is
- What a DAG is
- Why topological sort only works on directed acyclic graphs
- How to represent prerequisites as edges
- How to use indegree
- How Kahn's algorithm works
- How DFS topological sort works
- How to detect cycles with topological sort
- How to solve Course Schedule
- How to solve Course Schedule II
- How to solve Alien Dictionary
- How to solve Parallel Courses
- How to solve eventual safe states
- How topological sort connects to dependency ordering
- Common edge direction mistakes
- Common cycle detection mistakes

Core problems covered in this chapter:

1. Course Schedule
 2. Course Schedule II
 3. Find Eventual Safe States
 4. Alien Dictionary
 5. Parallel Courses
 6. Minimum Height Trees, contrast
 7. Sort Items by Groups Respecting Dependencies
 8. Sequence Reconstruction
 9. Build a Matrix With Conditions
 10. Loud and Rich
 11. Largest Color Value in a Directed Graph
 12. Recipe from Supplies
 13. All Ancestors of a Node in a DAG
 14. Course Schedule IV
 15. Longest Path in DAG
 16. Topological DP preview
-

1. What Is Topological Sort?

Topological sort orders nodes in a directed graph so that every directed edge is respected.

If there is an edge:

$u \rightarrow v$

then u must appear before v in the ordering.

Example:

$0 \rightarrow 1$

$0 \rightarrow 2$

$1 \rightarrow 3$

$2 \rightarrow 3$

Valid topological order:

0, 1, 2, 3

Another valid order:

0, 2, 1, 3

Both are valid because:

- 0 comes before 1
- 0 comes before 2
- 1 comes before 3
- 2 comes before 3

2. Why Topological Sort Exists

Many real problems involve dependencies.

Examples:

- take prerequisite courses before later courses
- build packages before packages that depend on them
- complete tasks before dependent tasks
- compile files in dependency order
- infer letter ordering from sorted alien words
- make recipes only after ingredients are available

Topological sort gives a valid order if dependencies do not contain a cycle.

3. What Is a DAG?

DAG means:

Directed Acyclic Graph

It has:

- directed edges
- no directed cycles

Example DAG:

A -> B -> C

A -> D

Not a DAG:

A -> B -> C -> A

This has a cycle.

If a cycle exists, there is no valid topological ordering because each node in the cycle depends on another node in the cycle.

4. Core Need / Why Use Topological Sort?

Use topological sort when you need an order that respects dependencies.

Example:

Prerequisites:

[course, prerequisite]

[1, 0]

[2, 1]

Means:

0 before 1

1 before 2

So edge:

0 -> 1

1 -> 2

Topological order:

0, 1, 2

Without topological sort, it is hard to know which nodes can safely be processed first.

5. When to Use Topological Sort

Use topological sort when the problem includes:

5.1 Prerequisites

Examples:

- Course Schedule
- Course Schedule II
- Course Schedule IV

5.2 Dependency Ordering

Examples:

- package installation
- build systems
- task scheduling

- recipe dependencies

5.3 Directed Acyclic Graph Ordering

Examples:

- all ancestors in DAG
- longest path in DAG
- topological DP

5.4 Cycle Detection in Directed Graph

Examples:

- can finish all courses?
- is dependency graph valid?
- is there circular dependency?

5.5 Lexicographic Ordering With Constraints

Example:

- Alien Dictionary

6. When NOT to Use Topological Sort

6.1 Undirected Graph Without Dependency Direction

Use DFS/BFS/Union Find.

6.2 Shortest Path in Unweighted Graph

Use BFS.

6.3 Weighted Shortest Path

Use Dijkstra or Bellman-Ford.

6.4 Need Connected Components

Use DFS/BFS/Union Find.

6.5 Graph Has Cycles and You Need Cycle Handling

Topological sort can detect cycle, but it cannot produce valid order if cycle exists.

7. Core Intuition

Topological sort repeatedly processes nodes with no remaining prerequisites.

A node with no prerequisites has:

indegree = 0

Indegree means number of incoming edges.

If:

$u \rightarrow v$

then v has one prerequisite from u .

Once u is processed, we reduce v 's indegree.

When v 's indegree becomes 0, it is ready.

8. Mental Models

8.1 Course Prerequisites

You can take a course only after all prerequisites are done.
Courses with no prerequisites can be taken first.

8.2 Dependency Unlocking

Completing one task unlocks dependent tasks.

8.3 Peeling Layers

Topological sort peels off nodes with indegree 0 layer by layer.

8.4 Cycle Trap

If no node has indegree 0 but nodes remain, those nodes are trapped in a cycle.

9. Pattern Recognition Checklist

Ask:

17. Are there directed dependencies?
18. Does one item need to come before another?
19. Does the problem ask if all tasks can be completed?
20. Does the problem ask for an order?
21. Does a cycle make completion impossible?
22. Can I model prerequisite as an edge?
23. Do I need process nodes with no prerequisites first?
24. Does indegree make sense?
25. Is this a DAG problem?
26. Is the graph directed?

If yes, topological sort may apply.

10. C++ Tools Needed

10.1 Adjacency List

```
vector<vector<int>>> graph(n);
```

10.2 Indegree Array

```
vector<int> indegree(n, 0);
```

10.3 Queue for Kahn's Algorithm

```
queue<int> q;
```

10.4 Result Order

```
vector<int> order;
```

10.5 DFS State Array

```
vector<int> state(n, 0);
```

State meanings:

0 = unvisited
1 = visiting
2 = done

11. Core Templates

Template 1 — Kahn's Algorithm

```
vector<int> topoSort(int n, vector<vector<int>>& edges) {  
    vector<vector<int>> graph(n);  
    vector<int> indegree(n, 0);  
  
    for (auto& edge : edges) {  
        int u = edge[0];  
        int v = edge[1];  
  
        graph[u].push_back(v);  
        indegree[v]++;  
    }  
  
    queue<int> q;  
  
    for (int node = 0; node < n; node++) {  
        if (indegree[node] == 0) {  
            q.push(node);  
        }  
    }  
  
    vector<int> order;  
  
    while (!q.empty()) {  
        int node = q.front();  
        q.pop();  
  
        order.push_back(node);  
  
        for (int neighbor : graph[node]) {  
            indegree[neighbor]--;  
  
            if (indegree[neighbor] == 0) {  
                q.push(neighbor);  
            }  
        }  
    }  
}
```

```

    }
}

if (order.size() != n) {
    return {};
}

return order;
}

```

Template 2 — Course Schedule Edge Build

For prerequisite pair:

[course, prereq]

Edge:

prereq -> course

C++:

```

for (auto& pre : prerequisites) {
    int course = pre[0];
    int prereq = pre[1];

    graph[prereq].push_back(course);
    indegree[course]++;
}

```

Template 3 — DFS Cycle Detection

```

bool hasCycle(int node,
              vector<vector<int>>& graph,
              vector<int>& state) {
    if (state[node] == 1) {
        return true;
    }

    if (state[node] == 2) {
        return false;
    }

    state[node] = 1;

    for (int neighbor : graph[node]) {
        if (hasCycle(neighbor, graph, state)) {

```

```

        return true;
    }
}

state[node] = 2;

return false;
}

```

Template 4 — DFS Topological Sort

```

bool dfs(int node,
        vector<vector<int>>& graph,
        vector<int>& state,
        vector<int>& order) {
    if (state[node] == 1) {
        return false;
    }

    if (state[node] == 2) {
        return true;
    }

    state[node] = 1;

    for (int neighbor : graph[node]) {
        if (!dfs(neighbor, graph, state, order)) {
            return false;
        }
    }

    state[node] = 2;
    order.push_back(node);

    return true;
}

vector<int> topoSortDFS(int n, vector<vector<int>>& graph) {
    vector<int> state(n, 0);
    vector<int> order;

    for (int node = 0; node < n; node++) {
        if (state[node] == 0) {

```

```

        if (!dfs(node, graph, state, order)) {
            return {};
        }
    }
}

reverse(order.begin(), order.end());

return order;
}

```

Template 5 — Topological Sort With Min-Heap

Use this when multiple valid orders exist and you need smallest lexicographic order.

```

priority_queue<int, vector<int>, greater<int>> pq;

for (int node = 0; node < n; node++) {
    if (indegree[node] == 0) {
        pq.push(node);
    }
}

while (!pq.empty()) {
    int node = pq.top();
    pq.pop();

    order.push_back(node);

    for (int neighbor : graph[node]) {
        indegree[neighbor]--;

        if (indegree[neighbor] == 0) {
            pq.push(neighbor);
        }
    }
}

```

12. Kahn's Algorithm Step-by-Step

Given:

```

0 -> 1
0 -> 2

```

1 -> 3
2 -> 3

Indegrees:

0: 0
1: 1
2: 1
3: 2

Queue starts:

[0]

Process 0:

1 indegree becomes 0
2 indegree becomes 0

Queue:

[1, 2]

Process 1:

3 indegree becomes 1

Process 2:

3 indegree becomes 0

Process 3.

Order:

0, 1, 2, 3

13. Cycle Detection With Kahn's Algorithm

If a graph has a cycle:

0 -> 1 -> 2 -> 0

Indegrees:

0: 1
1: 1
2: 1

No node has indegree 0.

Queue is empty.

Order remains empty.

Since:

order.size() != n

cycle exists.

This means no topological ordering is possible.

14. Kahn vs DFS Topological Sort

Main tool	indegree + queue	recursion + state
Cycle detection	processed count < n	visiting state reached
Order construction	push when indegree 0	push after children
Easier for courses	yes	also works
Easier for cycle state	sometimes DFS	yes
Lexicographic order	use min-heap	harder
Iterative	naturally	recursion

Use Kahn's algorithm when the problem is about prerequisites and processing ready nodes.

Use DFS when cycle detection or postorder dependency logic feels natural.

15. Major Variations

Can finish tasks	topo count == n	Course Schedule
Return order	Kahn order	Course Schedule II
Detect directed cycle	3-state DFS	Course Schedule
Lexicographic topo	min-heap	Alien Dictionary variant
Reverse graph topo	outdegree peeling	Eventual Safe States
DAG DP	process topo order	Longest Path DAG
Multiple dependency groups	topo sort twice	Sort Items by Groups
Matrix constraints	row topo + col topo	Build Matrix

16. Full Problem Walkthrough 1 — LeetCode 207: Course Schedule

16.1 Problem Summary

There are numCourses courses.

Prerequisite pair:

[a, b]

means to take course a, you must first take course b.

Return true if you can finish all courses.

16.2 How to Identify Topological Sort

Courses are nodes.

Prerequisites are directed edges.

If there is a cycle, impossible.

Example cycle:

0 requires 1

1 requires 0

No course can be taken first.

16.3 Edge Direction

Pair:

[course, prerequisite]

Edge:

prerequisite -> course

So:

```
graph[prereq].push_back(course);  
indegree[course]++;
```

16.4 C++ Code — Kahn's Algorithm

```
class Solution {  
public:  
    bool canFinish(int numCourses, vector<vector<int>>& prerequisites) {  
        vector<vector<int>> graph(numCourses);  
        vector<int> indegree(numCourses, 0);  
  
        for (auto& pre : prerequisites) {  
            int course = pre[0];  
            int prereq = pre[1];  
  
            graph[prereq].push_back(course);  
            indegree[course]++;  
        }  
  
        queue<int> q;
```

```

    for (int course = 0; course < numCourses; course++) {
        if (indegree[course] == 0) {
            q.push(course);
        }
    }

    int taken = 0;

    while (!q.empty()) {
        int course = q.front();
        q.pop();

        taken++;

        for (int next : graph[course]) {
            indegree[next]--;

            if (indegree[next] == 0) {
                q.push(next);
            }
        }
    }

    return taken == numCourses;
}
};

```

16.5 Complexity

Let:

$V = \text{numCourses}$

$E = \text{prerequisites.size()}$

Time:

$O(V + E)$

Space:

$O(V + E)$

16.6 Common Mistakes

- Reversing edge direction
- Using undirected graph
- Not checking processed count

- Thinking one valid queue order is the only possible order
-

17. Full Problem Walkthrough 2 — LeetCode 210: Course Schedule II

17.1 Problem Summary

Return an order to take all courses.
If impossible, return empty array.

17.2 How to Identify Topological Sort

Same as Course Schedule, but now return actual ordering.

17.3 C++ Code

```
class Solution {
public:
    vector<int> findOrder(int numCourses, vector<vector<int>>& prerequisites) {
        vector<vector<int>> graph(numCourses);
        vector<int> indegree(numCourses, 0);

        for (auto& pre : prerequisites) {
            int course = pre[0];
            int prereq = pre[1];

            graph[prereq].push_back(course);
            indegree[course]++;
        }

        queue<int> q;

        for (int course = 0; course < numCourses; course++) {
            if (indegree[course] == 0) {
                q.push(course);
            }
        }

        vector<int> order;

        while (!q.empty()) {
            int course = q.front();
            q.pop();
```

```
        order.push_back(course);

        for (int next : graph[course]) {
            indegree[next]--;

            if (indegree[next] == 0) {
                q.push(next);
            }
        }

        if (order.size() != numCourses) {
            return {};
        }

        return order;
    }
};
```

17.4 Complexity

Time:

$O(V + E)$

Space:

$O(V + E)$

18. Full Problem Walkthrough 3 — LeetCode 802: Find Eventual Safe States

18.1 Problem Summary

A node is eventually safe if every possible path starting from it eventually ends at a terminal node.

Terminal node:

outdegree = 0

Return all safe nodes sorted ascending.

18.2 Topological Sort Insight

This problem can be solved with DFS cycle detection.

It can also be solved with reverse graph topological peeling.

Unsafe nodes are nodes that can reach cycles.
Safe nodes eventually lead only to terminal nodes.

Reverse approach:

27. Build reverse graph.
 28. Track outdegree.
 29. Start with terminal nodes.
 30. Remove terminal/safe nodes backward.
 31. Nodes whose outdegree becomes 0 are safe.
-

18.3 C++ Code — Reverse Topological Sort

```
class Solution {
public:
    vector<int> eventualSafeNodes(vector<vector<int>>& graph) {
        int n = graph.size();

        vector<vector<int>> reverseGraph(n);
        vector<int> outdegree(n, 0);

        for (int node = 0; node < n; node++) {
            outdegree[node] = graph[node].size();

            for (int neighbor : graph[node]) {
                reverseGraph[neighbor].push_back(node);
            }
        }

        queue<int> q;

        for (int node = 0; node < n; node++) {
            if (outdegree[node] == 0) {
                q.push(node);
            }
        }

        vector<bool> safe(n, false);

        while (!q.empty()) {
            int node = q.front();
            q.pop();

            safe[node] = true;

            for (int prev : reverseGraph[node]) {
                outdegree[prev]--;
                if (outdegree[prev] == 0) {
                    q.push(prev);
                }
            }
        }

        return safe;
    }
};
```

```

        outdegree[prev]--;

        if (outdegree[prev] == 0) {
            q.push(prev);
        }
    }
}

vector<int> result;

for (int node = 0; node < n; node++) {
    if (safe[node]) {
        result.push_back(node);
    }
}

return result;
}
};

```

18.4 Complexity

Time:

$O(V + E)$

Space:

$O(V + E)$

18.5 Key Lesson

Sometimes topological sort starts from:

indegree 0

Sometimes reverse topological peeling starts from:

outdegree 0

It depends on what “ready” means.

19. Full Problem Walkthrough 4 — Alien Dictionary

19.1 Problem Summary

Given sorted list of words in an alien language, infer a valid character order.

Example:

```
["wrt", "wrf", "er", "ett", "rftt"]
```

Output could be:

```
"wertf"
```

19.2 How to Identify Topological Sort

The sorted word list gives ordering constraints between characters.

Compare adjacent words.

Find first different character.

If:

```
word1[i] != word2[i]
```

then:

```
word1[i] comes before word2[i]
```

Create directed edge:

```
word1[i] -> word2[i]
```

Then topologically sort characters.

19.3 Important Invalid Case

If longer word comes before its prefix:

```
["abc", "ab"]
```

This is invalid.

No valid alien order exists.

19.4 C++ Code

```
string alienOrder(vector<string>& words) {
    unordered_map<char, unordered_set<char>> graph;
    unordered_map<char, int> indegree;

    for (string& word : words) {
        for (char c : word) {
            if (!indegree.count(c)) {
                indegree[c] = 0;
            }
        }
    }

    for (int i = 0; i + 1 < words.size(); i++) {
        string a = words[i];
        string b = words[i + 1];
```

```

int len = min(a.size(), b.size());
bool foundDifference = false;

for (int j = 0; j < len; j++) {
    if (a[j] != b[j]) {
        char u = a[j];
        char v = b[j];

        if (!graph[u].count(v)) {
            graph[u].insert(v);
            indegree[v]++;
        }

        foundDifference = true;
        break;
    }
}

if (!foundDifference && a.size() > b.size()) {
    return "";
}

queue<char> q;

for (auto& [c, deg] : indegree) {
    if (deg == 0) {
        q.push(c);
    }
}

string order;

while (!q.empty()) {
    char c = q.front();
    q.pop();

    order.push_back(c);

    for (char next : graph[c]) {
        indegree[next]--;
    }
}

```

```

        if (indegree[next] == 0) {
            q.push(next);
        }
    }
}

if (order.size() != indegree.size()) {
    return "";
}

return order;
}

```

19.5 Complexity

Let total characters across all words be C .

Time:

$O(C)$

Space:

$O(1)$

if alphabet is fixed lowercase English, otherwise $O(\text{unique characters} + \text{edges})$.

19.6 Common Mistakes

- Comparing non-adjacent words unnecessarily
- Adding edges for every different character instead of first difference only
- Missing prefix invalid case
- Double-counting duplicate edges
- Forgetting isolated characters

20. Full Problem Walkthrough 5 — Parallel Courses

20.1 Problem Summary

There are n courses labeled $1..n$.

Relations:

prevCourse -> nextCourse

In one semester, you can take any number of courses with all prerequisites satisfied. Return minimum semesters needed to finish all courses.

If impossible, return -1.

20.2 How to Identify Topological Sort

Each semester corresponds to one BFS layer of currently available courses.
Use Kahn's algorithm level by level.

20.3 C++ Code

```
int minimumSemesters(int n, vector<vector<int>>& relations) {
    vector<vector<int>> graph(n + 1);
    vector<int> indegree(n + 1, 0);

    for (auto& relation : relations) {
        int u = relation[0];
        int v = relation[1];

        graph[u].push_back(v);
        indegree[v]++;
    }

    queue<int> q;

    for (int course = 1; course <= n; course++) {
        if (indegree[course] == 0) {
            q.push(course);
        }
    }

    int semesters = 0;
    int taken = 0;

    while (!q.empty()) {
        int levelSize = q.size();
        semesters++;

        for (int i = 0; i < levelSize; i++) {
            int course = q.front();
            q.pop();

            taken++;

            for (int next : graph[course]) {
                indegree[next]--;

                if (indegree[next] == 0) {
                    q.push(next);
                }
            }
        }
    }

    return semesters;
}
```

```

    }
    }
    }
    }

    return taken == n ? semesters : -1;
}

```

20.4 Complexity

Time:

$O(V + E)$

Space:

$O(V + E)$

20.5 Key Lesson

Topological BFS levels can represent time steps when all currently available tasks can be done in parallel.

21. Full Problem Walkthrough 6 — LeetCode 310: Minimum Height Trees, Contrast

21.1 Problem Summary

Given an undirected tree, find all roots that produce minimum height.

21.2 Why This Looks Like Topological Sort

This problem uses a similar peeling idea.

But it is not a directed dependency graph.

It peels leaves from an undirected tree until 1 or 2 centers remain.

This is degree-based BFS, not classic topological sort.

21.3 C++ Code

```

class Solution {
public:
    vector<int> findMinHeightTrees(int n, vector<vector<int>>& edges) {
        if (n == 1) {
            return {0};
        }

        vector<vector<int>> graph(n);

```

```

vector<int> degree(n, 0);

for (auto& edge : edges) {
    int u = edge[0];
    int v = edge[1];

    graph[u].push_back(v);
    graph[v].push_back(u);

    degree[u]++;
    degree[v]++;
}

queue<int> q;

for (int node = 0; node < n; node++) {
    if (degree[node] == 1) {
        q.push(node);
    }
}

int remaining = n;

while (remaining > 2) {
    int levelSize = q.size();
    remaining -= levelSize;

    for (int i = 0; i < levelSize; i++) {
        int leaf = q.front();
        q.pop();

        for (int neighbor : graph[leaf]) {
            degree[neighbor]--;

            if (degree[neighbor] == 1) {
                q.push(neighbor);
            }
        }
    }
}

vector<int> result;

```

```
while (!q.empty()) {
    result.push_back(q.front());
    q.pop();
}

return result;
}
};
```

21.4 Key Lesson

Not every degree-based queue problem is topological sort.

Classic topological sort is for directed graphs.

Minimum Height Trees is undirected leaf trimming.

22. Full Problem Walkthrough 7 — LeetCode 1203: Sort Items by Groups Respecting Dependencies

22.1 Problem Summary

There are n items and m groups.

Some items belong to groups.

Some dependencies say item $beforeItems[i]$ must come before item i .

Need return an ordering of all items such that:

- item dependencies are respected
 - items in the same group appear together
-

22.2 How to Identify Topological Sort

Need order groups and order items.

This is two-level topological sort:

32. Topologically sort groups.
 33. Topologically sort items within groups.
 34. Output items according to group order.
-

22.3 Simplified Strategy

Assign unique group IDs to ungrouped items.

Then build:

- item graph
- group graph

For dependency:

before -> item

If same group:

item-level dependency only

If different groups:

group-level dependency too

22.4 Core Topo Helper

```
vector<int> topo(vector<vector<int>>& graph, vector<int>& indegree) {
    queue<int> q;

    for (int i = 0; i < graph.size(); i++) {
        if (indegree[i] == 0) {
            q.push(i);
        }
    }

    vector<int> order;

    while (!q.empty()) {
        int node = q.front();
        q.pop();

        order.push_back(node);

        for (int nei : graph[node]) {
            indegree[nei]--;

            if (indegree[nei] == 0) {
                q.push(nei);
            }
        }
    }

    if (order.size() != graph.size()) {
        return {};
    }

    return order;
}
```

22.5 Key Lesson

Some problems require multiple topological sorts over different abstraction levels.

23. Full Problem Walkthrough 8 — Sequence Reconstruction

23.1 Problem Summary

Given original sequence and list of subsequences, determine if original sequence can be uniquely reconstructed.

23.2 How to Identify Topological Sort

Subsequences define ordering constraints.

Need know whether topological order is unique.

Uniqueness condition:

At every step, there must be exactly one node with indegree 0.

If there are multiple choices, multiple valid orders exist.

23.3 Key Logic

```
if (q.size() > 1) {  
    return false;  
}
```

Also compare produced order with original.

23.4 C++ Sketch

```
bool sequenceReconstruction(vector<int>& nums, vector<vector<int>>&  
sequences) {  
    unordered_map<int, unordered_set<int>> graph;  
    unordered_map<int, int> indegree;  
  
    for (int x : nums) {  
        graph[x];  
        indegree[x] = 0;  
    }  
  
    for (auto& seq : sequences) {  
        for (int x : seq) {  
            if (!indegree.count(x)) {  
                return false;  
            }  
        }  
    }  
  
    for (int i = 1; i < seq.size(); i++) {  
        int u = seq[i - 1];  
        int v = seq[i];
```

```

        if (!graph[u].count(v)) {
            graph[u].insert(v);
            indegree[v]++;
        }
    }
}

queue<int> q;

for (auto& [node, deg] : indegree) {
    if (deg == 0) {
        q.push(node);
    }
}

vector<int> order;

while (!q.empty()) {
    if (q.size() > 1) {
        return false;
    }

    int node = q.front();
    q.pop();

    order.push_back(node);

    for (int nei : graph[node]) {
        indegree[nei]--;

        if (indegree[nei] == 0) {
            q.push(nei);
        }
    }
}

return order == nums;
}

```

24. Full Problem Walkthrough 9 — LeetCode 2392: Build a Matrix With Conditions

24.1 Problem Summary

Given integer k .

Need build $k \times k$ matrix containing numbers $1..k$.

Row conditions:

a before b

means a must be in a row above b.

Column conditions:

a before b

means a must be in a column left of b.

Return matrix or empty if impossible.

24.2 How to Identify Topological Sort

Rows require one topological order.

Columns require another topological order.

If either has cycle, impossible.

24.3 C++ Code

```
class Solution {
public:
    vector<int> topo(int k, vector<vector<int>>& conditions) {
        vector<vector<int>> graph(k + 1);
        vector<int> indegree(k + 1, 0);

        for (auto& condition : conditions) {
            int u = condition[0];
            int v = condition[1];

            graph[u].push_back(v);
            indegree[v]++;
        }

        queue<int> q;

        for (int node = 1; node <= k; node++) {
            if (indegree[node] == 0) {
```

```

        q.push(node);
    }
}

vector<int> order;

while (!q.empty()) {
    int node = q.front();
    q.pop();

    order.push_back(node);

    for (int nei : graph[node]) {
        indegree[nei]--;

        if (indegree[nei] == 0) {
            q.push(nei);
        }
    }
}

if (order.size() != k) {
    return {};
}

return order;
}

vector<vector<int>> buildMatrix(int k,
                               vector<vector<int>>& rowConditions,
                               vector<vector<int>>& colConditions) {
    vector<int> rowOrder = topo(k, rowConditions);
    vector<int> colOrder = topo(k, colConditions);

    if (rowOrder.empty() || colOrder.empty()) {
        return {};
    }

    vector<int> rowPos(k + 1);
    vector<int> colPos(k + 1);

    for (int i = 0; i < k; i++) {

```

```

        rowPos[rowOrder[i]] = i;
        colPos[colOrder[i]] = i;
    }

    vector<vector<int>> matrix(k, vector<int>(k, 0));

    for (int value = 1; value <= k; value++) {
        matrix[rowPos[value]][colPos[value]] = value;
    }

    return matrix;
}
};

```

24.4 Complexity

Let:

R = rowConditions.size()
C = colConditions.size()

Time:

$O(k + R + C)$

Space:

$O(k + R + C)$

25. Full Problem Walkthrough 10 — Loud and Rich

25.1 Problem Summary

If person a is richer than person b, then edge:

a -> b

For each person, find the quietest person among everyone who is richer than or equal to them.

25.2 Topological Idea

Richer people should propagate their best quiet answer to poorer people.

Build edges:

richer -> poorer

Start with indegree 0 richest sources.

Process in topological order.

25.3 C++ Code

```
class Solution {
public:
    vector<int> loudAndRich(vector<vector<int>>& richer, vector<int>& quiet) {
        int n = quiet.size();

        vector<vector<int>> graph(n);
        vector<int> indegree(n, 0);

        for (auto& edge : richer) {
            int rich = edge[0];
            int poor = edge[1];

            graph[rich].push_back(poor);
            indegree[poor]++;
        }

        vector<int> answer(n);

        for (int i = 0; i < n; i++) {
            answer[i] = i;
        }

        queue<int> q;

        for (int i = 0; i < n; i++) {
            if (indegree[i] == 0) {
                q.push(i);
            }
        }

        while (!q.empty()) {
            int person = q.front();
            q.pop();

            for (int poorer : graph[person]) {
                if (quiet[answer[person]] < quiet[answer[poorer]]) {
                    answer[poorer] = answer[person];
                }

                indegree[poorer]--;
            }
        }
    }
};
```

```

        if (indegree[poorer] == 0) {
            q.push(poorer);
        }
    }
}

return answer;
}
};

```

25.4 Complexity

Time:

$O(V + E)$

Space:

$O(V + E)$

26. Full Problem Walkthrough 11 — LeetCode 1857: Largest Color Value in a Directed Graph

26.1 Problem Summary

Given directed graph and color for each node.

A path's color value is the maximum frequency of any one color along that path.

Return largest color value.

If graph has cycle, return -1.

26.2 How to Identify Topological DP

Need process directed graph.

Cycle invalid.

For each node, maintain best count of each color along paths ending at that node.

Topological order ensures predecessors are processed before successors.

26.3 C++ Code

```

class Solution {
public:
    int largestPathValue(string colors, vector<vector<int>>& edges) {
        int n = colors.size();

        vector<vector<int>> graph(n);
        vector<int> indegree(n, 0);
    }
};

```

```

for (auto& edge : edges) {
    int u = edge[0];
    int v = edge[1];

    graph[u].push_back(v);
    indegree[v]++;
}

queue<int> q;

for (int node = 0; node < n; node++) {
    if (indegree[node] == 0) {
        q.push(node);
    }
}

vector<vector<int>> dp(n, vector<int>(26, 0));

int processed = 0;
int answer = 0;

while (!q.empty()) {
    int node = q.front();
    q.pop();

    processed++;

    int colorIndex = colors[node] - 'a';
    dp[node][colorIndex]++;

    answer = max(answer, dp[node][colorIndex]);

    for (int nei : graph[node]) {
        for (int c = 0; c < 26; c++) {
            dp[nei][c] = max(dp[nei][c], dp[node][c]);
        }

        indegree[nei]--;

        if (indegree[nei] == 0) {
            q.push(nei);
        }
    }
}

```

```

        }
    }
}

if (processed != n) {
    return -1;
}

return answer;
}
};

```

26.4 Complexity

Time:

$O((V + E) * 26)$

Space:

$O(V * 26 + E)$

27. Full Problem Walkthrough 12 — Recipes From Supplies

27.1 Problem Summary

Given recipes, ingredients for each recipe, and initial supplies.

A recipe can be made if all ingredients are available.

Made recipes become supplies for later recipes.

Return all recipes that can be made.

27.2 How to Identify Topological Sort

Ingredients unlock recipes.

Treat ingredient/supply names as nodes.

For each ingredient needed by recipe:

ingredient -> recipe

Recipe indegree is number of missing ingredients.

Start queue with initial supplies.

27.3 C++ Code

```

class Solution {
public:
    vector<string> findAllRecipes(vector<string>& recipes,
                                vector<vector<string>>& ingredients,

```

```

        vector<string>& supplies) {
    unordered_map<string, vector<string>> graph;
    unordered_map<string, int> indegree;
    unordered_set<string> recipeSet(recipes.begin(), recipes.end());

    for (int i = 0; i < recipes.size(); i++) {
        string recipe = recipes[i];
        indegree[recipe] = ingredients[i].size();

        for (string& ingredient : ingredients[i]) {
            graph[ingredient].push_back(recipe);
        }
    }

    queue<string> q;

    for (string& supply : supplies) {
        q.push(supply);
    }

    vector<string> result;

    while (!q.empty()) {
        string item = q.front();
        q.pop();

        for (string& recipe : graph[item]) {
            indegree[recipe]--;

            if (indegree[recipe] == 0) {
                result.push_back(recipe);
                q.push(recipe);
            }
        }
    }

    return result;
}
};

```

27.4 Complexity

Let total ingredients be I .

Time:

$O(I + S)$

where S is number of supplies.

Space:

$O(I)$

28. Mini Problem 1 — Detect Cycle With Kahn

```
bool hasCycle(int n, vector<vector<int>>& edges) {
    vector<vector<int>> graph(n);
    vector<int> indegree(n, 0);

    for (auto& edge : edges) {
        int u = edge[0];
        int v = edge[1];

        graph[u].push_back(v);
        indegree[v]++;
    }

    queue<int> q;

    for (int i = 0; i < n; i++) {
        if (indegree[i] == 0) {
            q.push(i);
        }
    }

    int processed = 0;

    while (!q.empty()) {
        int node = q.front();
        q.pop();

        processed++;

        for (int nei : graph[node]) {
            indegree[nei]--;
```

```

        if (indegree[nei] == 0) {
            q.push(nei);
        }
    }
}

return processed != n;
}

```

29. Mini Problem 2 — Return Any Topological Order

```

vector<int> getOrder(int n, vector<vector<int>>& edges) {
    vector<vector<int>> graph(n);
    vector<int> indegree(n, 0);

    for (auto& edge : edges) {
        graph[edge[0]].push_back(edge[1]);
        indegree[edge[1]]++;
    }

    queue<int> q;

    for (int i = 0; i < n; i++) {
        if (indegree[i] == 0) {
            q.push(i);
        }
    }

    vector<int> order;

    while (!q.empty()) {
        int node = q.front();
        q.pop();

        order.push_back(node);

        for (int nei : graph[node]) {
            indegree[nei]--;

            if (indegree[nei] == 0) {

```

```

        q.push(nei);
    }
}

if (order.size() != n) {
    return {};
}

return order;
}

```

30. Mini Problem 3 — Unique Topological Order

```

bool hasUniqueTopoOrder(int n, vector<vector<int>>& edges) {
    vector<vector<int>> graph(n);
    vector<int> indegree(n, 0);

    for (auto& edge : edges) {
        graph[edge[0]].push_back(edge[1]);
        indegree[edge[1]]++;
    }

    queue<int> q;

    for (int i = 0; i < n; i++) {
        if (indegree[i] == 0) {
            q.push(i);
        }
    }

    int processed = 0;

    while (!q.empty()) {
        if (q.size() > 1) {
            return false;
        }

        int node = q.front();
        q.pop();
    }
}

```

```
processed++;

for (int nei : graph[node]) {
    indegree[nei]--;

    if (indegree[nei] == 0) {
        q.push(nei);
    }
}

return processed == n;
}
```

31. Common Edge Cases for Topological Sort

31.1 No Dependencies

Every node has indegree 0.

Any order is valid.

31.2 Single Node

Order is just that node.

31.3 Cycle

No valid ordering.

Return false, -1, or empty result depending on problem.

31.4 Disconnected DAG

Topological sort still works.

There may be multiple valid orders.

31.5 Duplicate Edges

Duplicate edges can incorrectly increase indegree multiple times.

Use set if duplicates are possible and should be ignored.

31.6 Isolated Nodes

Include them in result.

They have indegree 0.

31.7 1-Indexed Nodes

Use graph size $n + 1$.

32. Common Mistakes and Wrong Code

Mistake 1 — Wrong Edge Direction

For Course Schedule:

Wrong:

```
graph[course].push_back(prereq);
indegree[prereq]++;
```

Correct:

```
graph[prereq].push_back(course);
indegree[course]++;
```

Mistake 2 — Forgetting Cycle Check

Wrong:

```
return order;
```

Correct:

```
if (order.size() != n) {
    return {};
}
```

Mistake 3 — Treating Undirected Graph as Topological Sort

Topological sort is for directed graphs.

Mistake 4 — Missing Isolated Nodes

Need initialize all nodes, even if they have no edges.

Mistake 5 — Double Counting Duplicate Edges

If input can contain duplicate constraints, protect with a set.

Mistake 6 — DFS Topo Without Reversing

DFS postorder pushes node after children.

Need reverse at the end for standard topological order.

33. Interview Explanation Templates

33.1 Topological Sort

I model dependencies as directed edges. Then I use topological sort to process nodes only after all prerequisites are satisfied.

33.2 Kahn's Algorithm

I compute indegree for each node. Nodes with indegree 0 have no remaining prerequisites, so I put them in a queue. When I process a node, I reduce the indegree of its neighbors. If a neighbor reaches indegree 0, it becomes ready.

33.3 Cycle Detection

If after Kahn's algorithm I processed fewer than all nodes, the remaining nodes must be in a cycle or depend on a cycle, so no valid ordering exists.

33.4 Course Schedule

Each prerequisite pair [course, prereq] means prereq must come before course, so I add edge prereq -> course.

33.5 Alien Dictionary

I compare adjacent words and use the first differing character to create an ordering edge. Then I topologically sort characters. If a word appears before its own prefix, the input is invalid.

34. Complexity Patterns

Basic topological sort	$O(V + E)$	$O(V + E)$
Course Schedule	$O(V + E)$	$O(V + E)$
Course Schedule II	$O(V + E)$	$O(V + E)$
DFS cycle detection	$O(V + E)$	$O(V + E)$
Alien Dictionary	$O(\text{total characters})$	$O(\text{unique chars} + \text{edges})$
Parallel Courses	$O(V + E)$	$O(V + E)$
Eventual Safe States	$O(V + E)$	$O(V + E)$
Build Matrix Conditions	$O(k + R + C)$	$O(k + R + C)$
Loud and Rich	$O(V + E)$	$O(V + E)$
Largest Color Value	$O(26 * (V + E))$	$O(26 * V + E)$
Recipes From Supplies	$O(\text{total ingredients} + \text{supplies})$	$O(\text{total ingredients})$

35. Time-Boxed Practice Guidance

Easy / Intro Topological Sort

Target:

20–30 minutes

Examples:

- Course Schedule
- Course Schedule II

- Parallel Courses

Medium Topological Sort

Target:

30–50 minutes

Examples:

- Find Eventual Safe States
- Alien Dictionary
- Recipes From Supplies
- Loud and Rich

Harder Topological Sort

Target:

50–90 minutes

Examples:

- Sort Items by Groups Respecting Dependencies
- Build Matrix With Conditions
- Largest Color Value in a Directed Graph
- Sequence Reconstruction

36. Expanded Practice Set

36.1 Must-Do Medium

35. Course Schedule
36. Course Schedule II
37. Find Eventual Safe States
38. Alien Dictionary
39. Parallel Courses
40. Recipes From Supplies
41. Loud and Rich
42. Build a Matrix With Conditions

36.2 Strong Interview Practice

43. Sequence Reconstruction
44. Sort Items by Groups Respecting Dependencies
45. Largest Color Value in a Directed Graph
46. All Ancestors of a Node in a DAG
47. Course Schedule IV
48. Minimum Height Trees, contrast
49. Longest Path in DAG
50. Alien Dictionary with lexicographically smallest order

36.3 Hard / Stretch

51. Sort Items by Groups Respecting Dependencies
52. Largest Color Value in a Directed Graph
53. Alien Dictionary
54. Course Schedule IV
55. Parallel Courses III
56. Build Matrix With Conditions
57. Find All Possible Recipes From Given Supplies

37. Suggested Study Order

Step 1 — Basic Kahn's Algorithm

Start with:

- 59. Course Schedule
- 60. Course Schedule II

Goal:

Understand indegree and queue.

Step 2 — DFS Cycle Detection

Solve:

- 61. Course Schedule with DFS
- 62. Find Eventual Safe States with DFS

Goal:

Understand 3-state directed cycle detection.

Step 3 — Dependency Ordering Variants

Solve:

- 63. Parallel Courses
- 64. Recipes From Supplies
- 65. Loud and Rich

Goal:

Learn how dependencies propagate.

Step 4 — Character / Constraint Ordering

Solve:

- 66. Alien Dictionary
- 67. Sequence Reconstruction
- 68. Build Matrix With Conditions

Goal:

Extract ordering constraints from problem statements.

Step 5 — Advanced Topological DP

Solve:

- 69. Largest Color Value
- 70. Longest Path in DAG
- 71. Course Schedule IV

Goal:

Use topological order for dynamic programming.

38. Pattern Decision Table

can finish courses	Kahn count or DFS cycle	Course Schedule

Problem Clue	Topological Technique	Example
return course order	Kahn order	Course Schedule II
prerequisites	directed edge prereq -> course	Course Schedule
terminal safe nodes	reverse graph + outdegree	Safe States
alien alphabet	first differing char edge	Alien Dictionary
semesters/layers	BFS levels of indegree 0	Parallel Courses
unique reconstruction	queue size must be 1	Sequence Reconstruction
row and column constraints	topo twice	Build Matrix
recipe dependencies	supplies as sources	Recipes
propagate best value	topo DP	Loud and Rich
cycle invalid + path DP	topo DP with processed count	Largest Color Value

39. Final Cheat Sheet

Kahn's Algorithm

```

queue<int> q;

for (int i = 0; i < n; i++) {
    if (indegree[i] == 0) {
        q.push(i);
    }
}

while (!q.empty()) {
    int node = q.front();
    q.pop();

    order.push_back(node);

    for (int nei : graph[node]) {
        indegree[nei]--;
    }
}

```

```

        if (indegree[nei] == 0) {
            q.push(nei);
        }
    }
}

```

```

if (order.size() != n) {
    // cycle
}

```

Course Schedule Edge

```

int course = pre[0];
int prereq = pre[1];

graph[prereq].push_back(course);
indegree[course]++;

```

DFS Cycle State

```

0 = unvisited
1 = visiting
2 = done
if (state[node] == 1) {
    return true;
}

if (state[node] == 2) {
    return false;
}

```

Unique Topological Order

```

if (q.size() > 1) {
    return false;
}

```

Alien Dictionary First Difference

```

if (a[j] != b[j]) {
    graph[a[j]].insert(b[j]);
    break;
}

```

Prefix Invalid Case

```
if (!foundDifference && a.size() > b.size()) {  
    return "";  
}
```

40. What to Master Before Chapter 28

Before moving to Union Find, make sure you can:

- Explain topological sort
- Explain DAG
- Build directed graph from prerequisites
- Compute indegrees
- Use queue of indegree 0 nodes
- Detect cycle using processed count
- Return topological order
- Use DFS 3-state cycle detection
- Avoid wrong edge direction
- Handle disconnected DAGs
- Include isolated nodes
- Solve Course Schedule
- Solve Course Schedule II
- Solve Alien Dictionary
- Solve Eventual Safe States
- Understand topological BFS levels
- Know when topological sort does not apply

41. Core Takeaway

Topological sort orders directed dependencies.

The core rule is:

If $u \rightarrow v$, then u must come before v .

Kahn's algorithm works by repeatedly taking nodes with:

indegree = 0

These nodes have no remaining prerequisites.

If all nodes are processed, a valid order exists.

If not, there is a cycle.

Topological sort is the right pattern whenever the problem asks:

Can I complete all tasks?

In what order should dependencies be processed?

Does a directed dependency cycle exist?

The most important implementation detail is edge direction.

For prerequisites:

prerequisite $\rightarrow$ dependent task

Get that right, and most topological sort problems become much easier.